

Predicción de la Edad de un Abulón a través de Machine Learning

Jorge Manuel Oyoqui Aguilera || A01711783

Inteligencia artificial avanzada para la ciencia de datos

Grupo 601

Profesor Benjamín Valdés Aguirre

30/08/2026

I. Introducción

1.1. Objetivo

El objetivo principal de mi modelo fue lograr predecir la edad de un abulón a través de sus características físicas implementando manualmente todo el proceso de aprendizaje de dicho modelo sin usar ningún framework o biblioteca de Machine Learning. Pero primero entendamos el problema, empezando por saber por qué nos interesaría conocer la edad de un abulón.

1.2. ¿Por qué predecir la edad de un abulón?

Compartiendo algo de contexto, los abulones son moluscos marinos en peligro de extinción que se encuentran en aguas costeras frías de todo el mundo, distribuidos frente a las costas de Nueva Zelanda, Sudáfrica, Australia, el oeste de Norteamérica y Japón. Son alimentos muy nutritivos, elegantes y poco comunes, y se consumen ampliamente en Francia, Nueva Zelanda, ciertas partes de Latinoamérica, Japón y Corea. Las conchas de abulón se utilizan con fines decorativos, por lo que el abulón tiene una gran importancia económica. El precio del abulón está directamente relacionado con su edad. Sin embargo, determinar la edad del abulón es un proceso muy difícil. Se forman anillos en la concha interna del abulón, uno por año. La edad del abulón se determina cortando la concha por la mitad, tiñéndola y contando el número de anillos con un microscopio.

Dado que algunos anillos son difíciles de detectar con este método, tradicionalmente se suma 1,5 al recuento de anillos como una aproximación razonable de la edad del abulón. Conocer el precio absoluto del abulón es importante tanto para los agricultores como para los consumidores, mientras que conocer la edad correcta es importante para los ambientalistas que buscan proteger esta especie en peligro de extinción.

Ahora, me gustaría aclarar que en mi dataset no había ninguna variable llamada Edad pero, como justamente ya mencioné, la forma más cercana de estimar la edad de un abulón es a través de su número de anillos usando la fórmula:

$$Edad_abulón = Número_anillos + 1.5$$

Así que el objetivo de mi modelo va a ser estimar la cantidad de anillos en un abulón a través de sus características físicas.

1.3. Modelo de Machine Learning a usar

Como el objetivo de mi modelo fue predecir el número de anillos (Rings) de un abulón, tuve que decidir con qué tipo de modelo iba a trabajar.

Recordemos que existen dos tipos de aprendizaje en Machine Learning: el aprendizaje supervisado y el no supervisado. En el caso del aprendizaje supervisado, los modelos se entrenan usando un conjunto de datos en el que la respuesta correcta ya se conoce. Con el aprendizaje no supervisado, el conjunto de datos que se usó para el entrenamiento el modelo no contiene la respuesta correcta.

Como en mi dataset si poseo el valor correcto de la cantidad de anillos de cada abulón, entonces eso significaba que trabajaría con un modelo de **algoritmo de aprendizaje supervisado**.

Este tipo de algoritmos de aprendizaje supervisado se dividen en dos tipos de problemas: de regresión y de clasificación. Los problemas de clasificación son aquellos donde el resultado esperado es una categoría o clase, mientras que en los problemas de regresión lo que se espera es la predicción de un valor continuo.

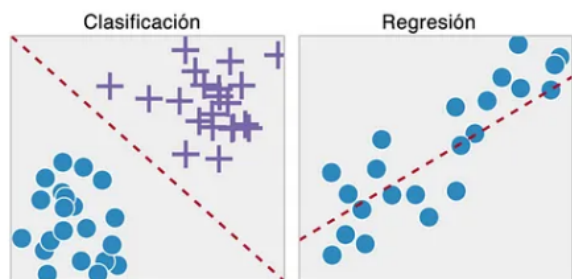


Figura 1. Algoritmos de clasificación y regresión

En mi caso, como voy a estar intentando predecir el valor de la variable Rings, es decir, un valor continuo, entonces el tipo de algoritmo que trabajaré para mi modelo fue uno de **regresión lineal múltiple** (múltiple debido a las múltiples variables independientes con las que trabajé).

Ahora, como no voy a poder utilizar frameworks, necesito un método de optimización para poder modificar y actualizar los pesos y el bias poco a poco para entrenar mi modelo y así reducir el MSE. Es ahí donde entra el Gradiente Descendente, con el cual decidí trabajar.

Así que, en general, con eso decidí que el modelo con el que iba a estar trabajando iba a ser un modelo de Regresión Lineal Múltiple con Gradiente Descendente.

Con eso claro, pasemos a la siguiente parte.

II. ETL

2.1. ¿Qué es ETL?

Cuando hablamos de Machine Learning, las siglas ETL significan extracción, transformación y carga de los datos. Este proceso es fundamental ya que es la fase para preparar, organizar y limpiar los datos que se usarán más adelante para entrenar y probar mi modelo. De esta forma, me aseguraré de tener datos limpios que el modelo pueda usar,

que no posea inconsistencias o sesgos y que se puedan usar para mi análisis.

2.2. Mi dataset

Para este proyecto, usé el dataset conocido como *abalone*, el cual se encuentra disponible en la siguiente liga: <https://archive.ics.uci.edu/dataset/1/abalone>.

El dataset proporciona la información de las características de los diferentes abulones que se estudiaron, las cuales serán usadas para predecir los valores de la variable *Rings*. Este dataset contiene en total 4177 instancias, cada una representando a un abulón diferente. Posee 8 características que describen las cualidades físicas de cada abulón más la variable objetivo *Rings*, siendo en total 9 columnas. A continuación, explico más detalladamente qué representa cada variable:

Característica	Descripción
Sex	El sexo del abulón. Se clasifica en M (male), F (female) e I (infant)
Length	Medida de la longitud máxima de la concha (en mm)
Diameter	Medida perpendicular a la longitud (en mm)
Height	La altura con la carne dentro de la concha (en mm)
Whole weight	Peso del abulón completo (en gramos)
Shucked weight	Peso de la carne sin la concha (en gramos)
Viscera weight	Peso del tracto digestivo tras el desangrado (en gramos)
Shell weight	Peso de la concha tras el secado (en gramos)
Rings	Número de anillos en la concha del abulón. +1.5 indica la edad en años del abulón (valor a predecir)

Tabla 1. Características de los abulones

2.3. Carga del dataset

Primero, descargué el dataset de UC Irvine Machine Learning Repository de la liga que antes compartí, para posteriormente usar la librería de pandas para cargar los datos en un dataframe llamado *df* y nombrar sus

columnas según correspondiera con la variable cuyos datos iba a contener.

2.4. Limpieza de los datos

2.4.1. Detección de valores inválidos

Luego de cargar los datos en el dataset, empecé a revisar los datos hasta que me di cuenta de que el valor mínimo de la variable Height era exactamente de cero. Eso no debía ser correcto, así que decidí investigar más a fondo y buscar todas las instancias con un valor de Height = 0. Al final, obtuve dos resultados. Tenía la hipótesis de que se iban a tratar de instancias donde todos sus valores fueran de cero, pero resultó no ser así. Los únicos datos con cero eran sus alturas. Como esos datos iban a ser problemáticos, al ser solamente dos instancias de 4177, decidí mejor eliminarlas al tratarse de sólo un 0.048% del total de mis datos.

2.4.2. Detección de valores nulos

Revisé si mi dataset poseía alguna instancia con valores nulos. Afortunadamente, ninguna tenía valores nulos, así que ahí no tuve que hacer ninguna limpieza.

2.4.3. Detección de outliers

Lo siguiente fue buscar posibles valores outliers en mi dataset. Para ello, generé una variable para guardar las columnas con un data type flotante o entero, sin contar a la columna de Rings, ya que se trata del target o variable objetivo. Posteriormente, generé una lista para guardar a los posibles outliers que fuera a encontrar.

Usé el rango intercuartílico de modo que si mi código llegaba a encontrar una instancia con algún valor que se encontrara en:

$$Cota_{inferior} = Q1 - 1.5 * IQR$$

o

$$Cota_{superior} = Q3 + 1.5 * IQR$$

sería tomado como outlier y almacenado en la lista anteriormente mencionada.

Al final, el número de posibles outliers encontrados fue de 151, así que lo siguiente por hacer fue decidir qué iba a hacer con esos outliers. Mi primera opción fue eliminarlos, pero consideré que estaría perdiendo varios datos, aún si sólo se trataba del 3.66% del total de datos (contando los dos registros con Height=0 que

eliminé previamente). Luego, me llegó la suposición de que dichos outliers podían tratarse de abulones o muy pequeños o muy grandes. Así que para comprobarlo, decidí revisar manualmente a los outliers y ver si sus datos eran consistentes entre sí.

Tras la revisión, noté que varios de los datos sí eran extremos, más no inconsistentes. Muchos se trataban o de abulones muy jóvenes cuyos datos eran pequeños debido a su temprana edad, y otros eran abulones muy viejos o grandes que también se alejaban del rango intercuartílico, pero no por eso eran errores.

Sólo hubo un caso que se me hizo extraño, que fue el de un abulón con índice 2051. Este poseía una altura de 1.13mm, el cual era del doble de su diámetro (el cual era de 0.355mm). Esto tendría que ser físicamente incorrecto. Pero a lo mejor podía ser un caso extremo, así que decidí comparar sus datos con los otros 9 abulones más altos de mi dataset. El abulón con índice 2051 fue el más alto de la lista, mientras que el segundo más alto apenas tenía la mitad de su tamaño, pero con un diámetro más grande. Con el resto de los datos fue parecido, de modo que Height tenía no sólo una inconsistencia con su altura, sino que con el resto de sus datos, siendo que el resto de sus características eran por mucho menores a las de los otros 9 abulones más altos. Esa inconsistencia con sus propias características me hizo reconocerlo como un outlier, y decidí eliminarlo de mi dataset. A continuación, muestro la tabla de comparación para que se pueda visualizar mejor la enorme diferencia e inconsistencia de la que hablo:

	Sex	Length	Diameter	Height	Whole	Rings
2051	F	0.455	0.355	1.130	0.5940	8
1417	M	0.705	0.565	0.515	2.2100	10
1763	M	0.775	0.630	0.250	2.7795	12
1428	F	0.815	0.650	0.250	2.2550	14
2179	F	0.595	0.470	0.250	1.2830	14
1528	M	0.725	0.575	0.240	2.2100	13
277	M	0.610	0.500	0.240	1.6420	18
307	M	0.645	0.515	0.240	1.5415	13
2161	F	0.715	0.565	0.240	2.1995	17
2212	F	0.595	0.495	0.235	1.3660	13

Figura 2. Comparación de los abulones más altos

Con ese registro eliminado, me decidí a seguir trabajando con el resto de los abulones considerados posibles outliers para que mi modelo también supiera cómo trabajar con casos extremos, pues, así como mencioné, se trataban de datos extremos pero consistentes. Por eso los dejé. Y con eso terminé la fase de ETL y de limpieza de los datos.

III. Análisis de correlaciones

3.1. One-Hot Encoding de la variable Sex

Antes de dividir el dataset en conjuntos X y Y, al tener un modelo de regresión lineal, no puedo pasarle datos no numéricos al modelo, pero justamente eso es lo que es Sex: una variable no numérica. Al tener 3 categorías distintas (Male, Female e Infant), no podía simplemente asignarle un número del 0-2 o del 1-3 a cada valor, ya que entonces les estaría asignando indirectamente un valor numérico. Por ejemplo, si definiera que Male=1 y Female=2, entonces Female tendría mayor peso que Male y eso no es correcto.

En su lugar, me decidí por transformarlos en valores binarios. Pero claro, tengo tres valores diferentes y no dos. Así que, para resolver esto, luego de convertir la variable Sex en binaria, quité una de las tres variables para evitar que hubiera redundancia entre las variables y el intercepto, de modo que quedaría implícita en las otras dos. En este caso, la variable eliminada, y que quedó como categoría de referencia, fue Sex_F (o sea, las abulones femeninas). Más adelante notarán que es por eso que aparecen Sex_M y Sex_I, pero no Sex_F.

Siendo más específico, al cambiar la variable Sex a binaria, conseguí una tabla parecida a la siguiente:

Sex	Sex_F	Sex_I	Sex_M
F	1	0	0
I	0	1	0
M	0	0	1

Tabla 2. Variable binaria Sex.

Pero, al hacer `drop_first=True`, eliminé la primera columna de la tabla, que en este caso fue Sex_F. Y, al usar la función `.get_dummies()` de pandas, transformé las variables categóricas en variables numéricas binarias, de modo que la tabla pasó a tener la siguiente forma:

Sex	Sex_I	Sex_M
F	0	0
I	1	0
M	0	1

Tabla 3. Variable binaria Sex luego de eliminar la primera columna.

Es decir, Sex_F no desapareció del dataset original, sino que se volvió una categoría de referencia, de modo que más

adelante las correlaciones de Sex_M y Sex_I se implementaron respecto a la categoría F.

3.2. Separando el dataset en target y variables

Ya con todas las variables siendo numéricas, dividí el dataset en conjuntos X (para el target) y Y (para el resto de las variables). Definí a Rings como mi target ya que es la variable a predecir, mientras que al resto de las características las dejé como variables independientes.

3.3. Cálculo y visualización de las correlaciones

La correlación es la medida estadística que permite saber cómo es que dos o más variables se relacionan entre sí. Esto es importante ya que necesito ver cómo es que las distintas características físicas del abulón se relacionan entre sí y con la variable Rings. Así que, para ello, calculé la matriz de correlaciones entre las variables usando la función `.corr()` de pandas. Posteriormente, y para facilitar el entendimiento de las correlaciones, creé un Heatmap que graficara los resultados.

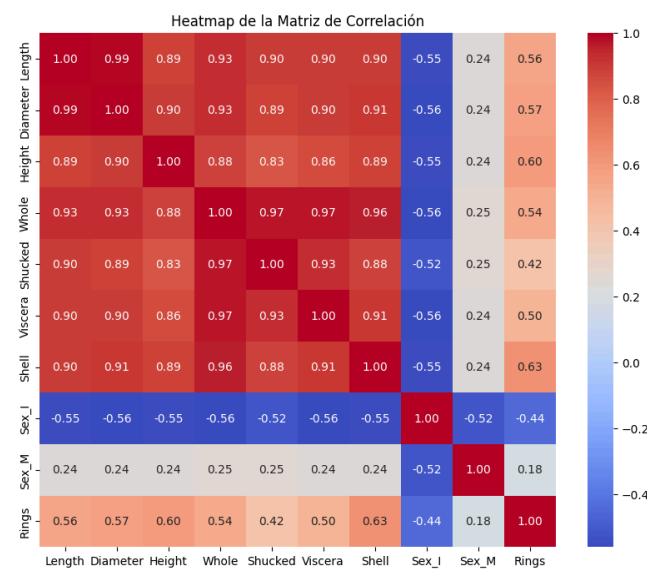


Figura 3. Heatmap de correlaciones

El mapa representa visualmente con colores las correlaciones entre las variables, donde el coeficiente de correlación puede tomar valores entre 1 a -1, siendo que 1.00 es la correlación positiva perfecta, -1.00 es la correlación negativa perfecta y 0.00 significa que no hay correlación.

En este caso, los colores representan lo siguiente:

- **Rojo:** Correlación positiva.
- **Azul:** Correlación negativa.
- **Blanco:** Correlación de 0 o sin correlación.

De modo que mientras más intenso sea el color, mayor es la magnitud de correlación.

Noté que la correlación entre las primeras siete variables (que van de Length hasta Shell) era extremadamente alta, con valores que iban del 83% hasta el 99% de correlación. Es decir, esa información está muy relacionada. Y de hecho eso hace sentido, ya que si un abulón es más grande, va a tener mayor peso, longitud, diámetro y altura. Aunque igualmente eso podría indicar que había multicolinealidad entre las variables independientes, y aunque eso no significa que no pueda usar la regresión lineal múltiple, sí va ser algo que afecte más adelante en los pesos individuales del modelo.

En el caso de Sex, podemos notar que Sex_I tuvo una asociación moderadamente negativa con el resto de las variables, mientras que Sex_M tiene una asociación más débil pero positiva. Y recordemos que todo esto es tomando implícitamente a Sex_F como categoría de referencia. O sea, ahora mismo la variable que estamos buscando predecir es la edad, y definimos al sexo femenino como nuestra categoría de referencia, por lo que se están comparando las variables de los abulones machos e infantes con las variables de las hembras. Por eso los abulones infantes tienen valores negativos, por que al ser más jóvenes, tienen menos edad y por ende son mucho más pequeños que el resto de las bulones hembras, mientras que los machos resultaron ser, en promedio, más viejos y grandes que las hembras, por eso sus valores son positivos pero más pequeños, por que la diferencia entre los macho y las hembras es pequeña.

En conclusión, todas las características físicas tienen correlaciones extremadamente fuertes que van aproximadamente de $0.83 \leq r \leq 0.99$; mientras que la correlación entre las variables físicas y Rings es más moderada, que va aproximadamente de $0.42 \leq r \leq 0.63$.

IV. Preparación de los datos para Machine Learning

4.1. División de los datos en conjuntos Train, Validation y Test

En el dataset actual, luego de la limpieza inicial, quedaron en total 4,174 instancias de abulones, pero la cuestión es que no podía mostrarle todos esos abulones al modelo ya que entonces no habría con qué probarlo, y solamente estaría memorizando la información, no aprendiendo a predecir el número de anillos. Así que, para evitar ese problema, dividí mis datos de la

siguiente forma: 70% se destinaron a entrenar el modelo, otro 15% para evaluar el rendimiento del modelo y para ajustar sus hiperparámetros y el último 15% van a ser reservados para probar y evaluar el modelo. Existen otras formas de dividir los datos, pero escogí esta debido a su equilibrio a la hora de dividir los datos.

Para lograr dividir los datos sin usar librería o frameworks de Machine Learning, mezclé los datos usando la librería Random (la cual no es de ML) para que la división de los datos fuera, justamente, aleatoria, y definí una semilla para siempre tener la misma semilla de datos, ya que no quería que con cada prueba mis resultados cambiaran. Definí la proporción de entrenamiento de 0.7 y lo multipliqué por el conjunto de datos ya mezclados, con lo cual pude obtener el 70% de los datos para el entrenamiento. Apliqué lo mismo para la proporción de validación (0.15) y de prueba (0.15). Finalmente, dividí mis conjuntos de datos X y Y usando los índices de entrenamiento y prueba, de modo que todos los datos en entrenamiento tuvieran los mismos índices en el mismo orden, y lo mismo para el conjunto de validación y de pruebas (porque de no hacerlo estaría combinando datos de diferentes índices y eso sería desastroso para el modelo).

Aunque en mi código no lo incluyo porque dejó de ser necesario, sí hice una comprobación para revisar si habían registros repetidos entre los conjuntos de entrenamiento, validación y prueba. Igualmente revisé que estuvieran todos los 4174 registros entre los tres conjuntos. Luego de esta comprobación, comprobé que efectivamente no se repetían índices de datos entre conjuntos y que tenía todos y cada uno de mis registros.

```
print(
    "¿Hay índices repetidos entre Train y Validation?",
    len(set(train_indices) & set(validation_indices)) > 0
)

print(
    "¿Hay índices repetidos entre Train y Test?",
    len(set(train_indices) & set(test_indices)) > 0
)

print(
    "¿Hay índices repetidos entre Validation y Test?",
    len(set(validation_indices) & set(test_indices)) > 0
)

¿Hay índices repetidos entre Train y Validation? False
¿Hay índices repetidos entre Train y Test? False
¿Hay índices repetidos entre Validation y Test? False
```

Figura 4. Comprobación de índices repetidos


```

all_indices = (
    set(train_indices)
    | set(validation_indices)
    | set(test_indices)
)

print(
    "¿Están todos los registros incluidos?",
    len(all_indices) == len(df_x_encoded)
)

¿Están todos los registros incluidos? True

```

Figura 5. Comprobación de que estén todos los registros

4.2. Estandarización de las variables

Antes de pasar al modelo de regresión lineal múltiple, tuve que hacer unos últimos cambios ya que la escala de mis datos era muy diferente entre sí. Los registros de las variables como Length iban de 0.075mm a 0.815mm, los de Whole de 0.002 a 2.8 gramos y las columnas de Sex iban de 0 a 1 sin intermedios. Tenía valores muy bajos, otros relativamente altos y otros eran simplemente binarios. Así que tuve que estandarizar la escala de mis datos para facilitar la optimización por parte del gradiente descendente.

El objetivo era transformar cada variable usando la fórmula de estandarización:

$$z = (x - \mu) / \sigma$$

Donde:

- x = valor original
- μ = Media de la variable
- σ = desviación estándar
- z = valor estandarizado

Pero esto no lo podía hacer directamente con todo el dataset de `df_x_encoded`, ya que eso haría que estuviera usando información de los datos de prueba para preparar los datos de entrenamiento. Es por eso que calculé la media y la desviación estándar usando solamente el conjunto de prueba, de modo que usé esos mismos datos en los conjuntos de validación y prueba también para estandarizar los conjuntos sin que el modelo pudiera ver el contenido del conjunto de pruebas.

Una vez estandarizados los conjuntos, los datos obtuvieron una media de aproximadamente 0 (ya que los valores reales fueron muy cercanos a cero) y con una desviación estándar de 1 para el conjunto de pruebas, y muy cercano a 1 para los conjuntos de validación y de prueba.

```

Media de X_train_scaled:
Length      2.067653e-16
Diameter    1.970351e-16
Height      -1.775749e-16
Whole       -9.243623e-17
Shucked     -1.568983e-16
Viscera     -3.405545e-17
Shell       1.459519e-16
Sex_I       1.143290e-16
Sex_M       2.432532e-17
dtype: float64

Desviación estándar de X_train_scaled:
Length      1.0
Diameter    1.0
Height      1.0
Whole       1.0
Shucked     1.0
Viscera     1.0
Shell       1.0
Sex_I       1.0
Sex_M       1.0
dtype: float64

```

Figura 6. Resultados de la estandarización para el conjunto de entrenamiento

```

Media de X_validation_scaled:
Length      -0.070937
Diameter    -0.076178
Height      -0.054271
Whole       -0.082448
Shucked     -0.088764
Viscera     -0.087231
Shell       -0.064062
Sex_I       0.070090
Sex_M       0.038190
dtype: float64

Desviación estándar de X_validation_scaled:
Length      0.987879
Diameter    0.990432
Height      0.955681
Whole       0.959896
Shucked     0.941455
Viscera     0.947334
Shell       0.997135
Sex_I       1.025504
Sex_M       1.010483
dtype: float64

```

Figura 7. Resultados de la estandarización para el conjunto de validación

```

Media de X_test_scaled:
Length      -0.020951
Diameter    -0.018624
Height      -0.044007
Whole       -0.036738
Shucked     -0.044493
Viscera     -0.038240
Shell       -0.025493
Sex_I       -0.007680
Sex_M       -0.036796
dtype: float64

Desviación estándar de X_test_scaled:
Length      0.985246
Diameter    0.986883
Height      0.952002
Whole       0.969769
Shucked     0.961544
Viscera     0.955480
Shell       0.974814
Sex_I       0.997565
Sex_M       0.989644
dtype: float64

```

Figura 8. Resultados de la estandarización para el conjunto de pruebas

A continuación, voy a explicar el por qué de estos resultados.

La media ≈ 0 es por que al restarle la media a cada valor de la variable al hacer $x - \mu$, desplacé toda la distribución de datos de modo que dicha distribución quedó centrada cerca de 0. Por eso los valores de la media son tan pequeños, no están exactamente en cero, pero sí moví la distribución para que estuviera centrada lo más cerca del 0 posible. Y luego, al dividir el resultado entre la desviación estándar (σ), cambié la escala de la variable de modo que sus unidades se mostraran en desviación estándar. De ahí el $\sigma=1$. Esto para que las variables quedaran en una escala comparable cuando llegara el momento de trabajar con el gradiente descendente.

V. Implementación del modelo de regresión lineal

5.1. Definición del modelo

Para poder predecir el número de anillos de un abulón, ya dije que trabajaría con la regresión lineal múltiple, de modo que iba a estar trabajando con la siguiente fórmula:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_9 x_9$$

donde:

- $\hat{y}$ = predicción
- x = característica
- β_0 = intercepto/bias
- β_1-9 = el resto de las 9 variables

Para mi regresión lineal múltiple trabajé con 9 características y , por ende, con 9 pesos más un intercepto, formando un total de 10 parámetros.

Ahora, no tenía ni conocía los pesos para β_0-9 (incluyendo el bias), así que el trabajo del gradiente descendente fue encontrar justamente los valores para esos pesos de modo que las predicciones fueran lo más cercanas posible a los Rings reales.

5.2. Inicialización de los parámetros

Para calcular los pesos podría haber tenido 9 variables diferentes que fueran de $\beta_1 = 0$ a $\beta_9 = 0$ pero, en vez de hacer eso, decidí ahorrarme tiempo creando una matriz que ya contuviese de $\beta_1-\beta_9$, de modo que el bias (β_0) quede separado.

Como aún no sabía cuales eran los mejores pesos (bias incluido), asumí que todos los pesos de los β eran iguales a 0. Es decir: $\beta_1=\beta_2=\dots=\beta_9=0$ y $\beta_0=0$.

5.3. Creación de la función de predicción

Realicé mi primera predicción de prueba. Recordemos que para ese momento mi conjunto de entrenamiento tenía una dimensión de 2921×9 , de los 2921 registros de mi dataset que fueron dedicados a entrenar el modelo, y de las 9 columnas con las características del abulón; por lo que también tenía 9 pesos. Entonces, para generar la primera predicción, hice una multiplicación entre matrices de la siguiente manera:

$$\hat{y} = XW$$

donde:

- X = son mis valores del conjunto de entrenamiento
- W = son los pesos

Lo que obtuve fue el producto entre las matrices de X y de pesos, de modo que hice prácticamente la siguiente operación:

$$\hat{y} = x_1 w_1 + x_2 w_2 + \dots + x_9 w_9 + bias$$

Así que, con eso implementado, realicé mis primeras predicciones donde, como inicialmente tanto los pesos como el bias son 0, mis predicciones también dieron 0 de resultado.

```
y_pred = predict(x_train_scaled, weights, bias)

print("Primeras predicciones:")
print(y_pred[:10])

Primeras predicciones:
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Figura 9. Resultados de mi primera predicción

5.4. Creación de la función de costo MSE

Ahora viene la duda del millón, y es: ¿cómo saber qué tan malas fueron las predicciones que hice? Ahora tenía las predicciones ' $\hat{y}$ ' y los valores reales de ' y '. Muy probablemente los valores reales de ' y ' no iban a ser 0, así que de seguro el modelo actual era pésimo. Para comprobarlo necesitaba medir dicho error

numéricamente. Para ello, implementé la función de costo MSE:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Donde:

- y_i = valor real
- $\hat{y}_i$ = predicción
- $y_i - \hat{y}_i$ = error
- $(y_i - \hat{y}_i)^2$ = error al cuadrado
- n = número de observaciones

Aclaración: La razón por la que elevamos el error al cuadrado es para evitar que se anulen los resultados positivos con los negativos. El objetivo es que el MSE sea lo más pequeño posible, ya que eso implica un menor error y, por ende, que el modelo está generando mejores predicciones.

Luego de implementar la función de costo, calculé el MSE de mi primera predicción, a lo que obtuve que MSE=109.0606. Como el modelo predijo 0 para todo, obtuve un error bastante grande. Esto ya me decía qué tan mal estaba mi modelo, más no me decía cómo debía corregir los pesos. No me decía si debía aumentar o disminuir algún peso. Para lograr actualizar los pesos necesitaba implementar el Gradiente Descendente, y eso es justo lo que hice más adelante, pero primero necesitaba implementar una cosa más.

5.5. Creación de la función de R^2

Una vez que ya sabía cuál era el MSE de mi modelo, también necesitaba saber qué tanto explicaba mi modelo la varianza con respecto a Rings. Para ello, también implementé la función de R^2 que me ayudará justamente para esa tarea. Tenemos que la fórmula de la R^2 es:

$$R^2 = 1 - \frac{MSE}{\sigma_y^2}$$

Donde:

- MSE = es el error cuadrático medio que calculamos previamente.
- σ_y^2 = la varianza de 'y'.

Luego de implementar la función del cuadrado del coeficiente de correlación, obtuve que $R^2 = -9.6559$. Al igual que con el caso del MSE, como el modelo predijo 0 para todo, mi R^2 me dijo que, de momento, mi modelo no explicaba en absoluto la proporción de la variación de Rings. Esto terminó de confirmarme lo que MSE ya me había dicho: el modelo era malo.

Así que el siguiente objetivo era actualizar los pesos y el bias para así poder reducir el MSE, de modo que eso también permitiría que el R^2 se hiciera mayor, representando así que el modelo explica más acerca de la variable de Rings.

5.6. Entendiendo la derivada del Gradiente Descendente

Ya conseguí el primer valor del MSE y del R^2 , los cuales fueron bastante malos. Ahora el objetivo era buscar valores para los pesos que minimizaran el valor del MSE y aumentaran el de R^2 . Por eso es que debía usar la derivada: para buscar dichos valores en los pesos que minimizaran lo más posible el valor de MSE.

Por poner un ejemplo, podemos imaginar que nuestra función nos posiciona en una colina. El objetivo era llegar a la base de dicha colina, que es donde estaría el MSE más pequeño, donde para eso probaría dar pasos en diferentes direcciones para poder ir bajando o subiendo poco a poco según lo que requiera para llegar a la base. Digamos que si estaba en una posición 5, debía bajar la colina; y si estaba en una posición -5, es porque estaba en una depresión y necesitaba subir. Esto de forma iterativa hasta encontrar la base de la colina.

Lo mismo se iba a hacer aquí. El objetivo era actualizar los pesos y sus direcciones (para ver si sus valores debían subir o bajar) y calcular su MSE de forma iterativa hasta encontrar aquellos valores que minimizaran lo más posible el MSE. Cabe aclarar que sería imposible llegar a un error de 0 (eso significaría que mi modelo se sobre ajustó a los datos), pero eso no significaba que no pudiera intentar aproximarme lo más posible.

En términos matemáticos, es lo mismo que hacer esto:

$$\frac{\partial MSE}{\partial w_1}$$

Esto representa cómo es el cambio en MSE cuando se modifica w_1 . Sería hacer eso para cada peso, incluyendo el bias:

$$\frac{\partial MSE}{\partial w_1} \cdots \frac{\partial MSE}{\partial w_9}$$

Con eso claro, el siguiente paso era derivar la gradiente. Actualmente tenía las siguientes fórmulas:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\hat{y}_i = b + \sum_{j=1}^9 w_j \cdot x_{ij}$$

Y el objetivo era obtener:

$$\frac{\partial MSE}{\partial w_j}$$

Para eso hice regla de la cadena, lo cual me llevó a las fórmulas de las derivadas de MSE con respecto a w_j y de MSE con respecto al bias. A continuación, describo el proceso matemático que realicé para llegar a dichas fórmulas.

1) Tenemos que el término que depende de w_j es $\hat{y}_i$, así que usé la regla de la cadena:

$$\frac{\partial MSE}{\partial w_j} = \frac{\partial MSE}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial w_j}$$

2) Derivé MSE con respecto a $\hat{y}_i$:

$$\frac{\partial MSE}{\partial \hat{y}_i} = \frac{1}{n} \cdot 2(\hat{y}_i - y_i) = \frac{2}{n} (\hat{y}_i - y_i)$$

Aclaración: No se toma en cuenta el símbolo Σ ya que esto se hace para cada término de la sumatoria.

3) Derivé y_i con respecto a w_j :

$$\frac{\partial \hat{y}_i}{\partial w_j} = 0 + 1 \cdot x_{ij} = x_{ij}$$

4) Multipliqué los resultados:

$$\frac{\partial MSE}{\partial w_j} = \frac{2}{n} \sum_{i=1}^n x_{ij} \cdot (\hat{y}_i - y_i)$$

5) Obtuve la derivada con respecto al bias, donde como es sesgo b afecta a todas las predicciones $\hat{y}_i$ por igual, entonces tengo que:

$$\frac{\partial \hat{y}_i}{\partial b} = 1$$

Por ende:

$$\frac{\partial MSE}{\partial b} = - \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Esto se da porque como el sesgo b no depende de i ni de ninguna características x_{ij} , al cambiar b , todas las predicciones cambian la misma cantidad. Por ejemplo, si b aumenta en 1, entonces cada $\hat{y}_i$ aumentará en 1 también. Por eso obtuve $\partial \hat{y}_i / \partial b = 1$.

Aclaración: El signo “-” en $\partial MSE / \partial b$ aparece al reescribir el término $(\hat{y}_i - y_i)$ a $-(y_i - \hat{y}_i)$. Esto se hace para expresar el gradiente en la dirección del descenso del error, que es la que se usa en el algoritmo del Gradiente Descendente.

Explicando mejor qué significa cada parte, el gradiente de un peso nos dice que tenemos $y_i - \hat{y}_i$, es decir, *error = valor real - valor predicción*. El x_{ij} es el valor que tenía la determinada característica del abulón. Así que al hacer $x_{ij} \cdot (y_i - \hat{y}_i)$, lo que estábamos diciendo es qué tan grande es el error $\times$ cuándo está involucrada esa característica en esa observación. Y eso se hizo para todos los abulones y así se obtuvo un valor que indicaba cómo es que afectaba modificar ese peso al MSE.

Es aquí donde entra la gradiente, ya que como tenía 9 pesos, iba a terminar teniendo nueve derivadas, y ese conjunto de derivadas iba a ser el gradiente. El gradiente se iba a utilizar para actualizar el valor de los pesos usando la siguiente fórmula:

$$w_j \leftarrow w_j - \alpha \left(\frac{\partial \text{MSE}}{\partial w_j} \right) \text{ para los pesos}$$

$$b \leftarrow b - \alpha \left(\frac{\partial \text{MSE}}{\partial b} \right) \text{ para el bias}$$

Donde α es el **learning rate**. Explicando más el concepto de learning rate, este es el hiper parámetro que definirá cuál será el tamaño de los pasos del modelo de Machine Learning al actualizar los pesos una vez se inicie su entrenamiento.

Importante: El Learning rate se debe escoger con cuidado, ya que un learning rate muy pequeño va a dar la respuesta correcta, pero va a tardar mucho en darla; mientras que un learning rate muy alto va a dar saltos demasiado grandes que compliquen encontrar los mejores valores de los pesos para disminuir el valor del MSE.

5.7. Implementación del gradiente descendente

Así que, con eso dicho, procedí a desarrollar la función que calculara el gradiente, a lo que luego de correrlo obtuve los siguientes resultados:

```
Gradiente de los pesos:
[-3.6169522 -3.71674899 -3.88337888 -3.47143064 -2.72286964 -3.25872175
-4.05483332 2.81163476 -1.16954822]

Gradiente del bias:
-19.88223211229031
```

Figura 10. Cálculo del primer gradiente

Como se puede observar, conseguí 10 gradientes debido a que tengo 9 pesos diferentes y un bias.

Definé un learning rate de 0.01 y usé los gradientes que obtuve previamente para actualizar los pesos, a lo que obtuve el siguiente resultado:

```
Nuevos pesos:
[ 0.03616952 0.03716749 0.03883379 0.03471431 0.02722287 0.03258722
 0.04054833 -0.02811635 0.01169548]

Nuevo bias:
0.1988223211229031
```

Figura 11. Nuevos pesos del modelo

Con esto mi modelo ya dió su primer paso de aprendizaje, así que ahora debo comprobar que realmente haya aprendido revisando que el MSE nuevo

sea menor al MSE viejo. Luego de comparar, vi que el nuevo MSE era de 104.2360, mientras que el viejo, cuando todos los pesos tenían un valor de 0, era de 109.0606. En el caso de R^2 , pasó de un valor inicial de -9.6559 a un valor de -9.1845. Eso significaba que la fórmula de gradiente funcionó correctamente, así que procedí a realizar el mismo proceso pero de forma iterativa y automática.

Pero antes de eso, tuve que definir cuántas iteraciones se iban a hacer. No podía simplemente dejar que el modelo iterara eternamente y esperar a que llegara a un MSE cercano a cero luego de una cantidad indefinida pero alta de iteraciones. En lugar de eso, definí la condición de que el máximo de iteraciones (o epochs) iban a ser 1000. Esto garantizaría que eventualmente las iteraciones fueran a terminar, sin importar el valor que tuviesen el MSE y el R^2 . Así que me mantuve con las 1000 iteraciones (epochs) y con un learning rate de 0.01.

Solo que antes de iniciar con la regresión, previamente ya había hecho una actualización manual para comprobar que mi función de gradiente funcionara, como ya pudieron leer anteriormente. Así que lo que hice fue reiniciar los pesos para que sus valores fueran nuevamente de 0, de modo que el entrenamiento se pudiera desarrollar de la forma más limpia posible.

Una vez construido el ciclo para actualizar los pesos, agregué un output que me devolviera el valor del MSE y otro que me devolviera el valor de R^2 cada 100 epochs para así documentar cómo es que descendía el error e incrementaba la proporción de la varianza de Rings explicada por el modelo. Luego de los 1000 epochs, conseguí los siguientes resultados:

```
Epoch 0: Train MSE = 109.0606, Validation MSE = 107.8676
Epoch 0: Train R^2 = -9.6559, Validation R^2 = -9.0349

Epoch 100: Train MSE = 7.5505, Validation MSE = 7.9745
Epoch 100: Train R^2 = 0.2623, Validation R^2 = 0.2581

Epoch 200: Train MSE = 5.3982, Validation MSE = 5.6756
Epoch 200: Train R^2 = 0.4726, Validation R^2 = 0.4720

Epoch 300: Train MSE = 5.1318, Validation MSE = 5.3672
Epoch 300: Train R^2 = 0.4986, Validation R^2 = 0.5007

Epoch 400: Train MSE = 4.9966, Validation MSE = 5.2089
Epoch 400: Train R^2 = 0.5118, Validation R^2 = 0.5154

Epoch 500: Train MSE = 4.9158, Validation MSE = 5.1128
Epoch 500: Train R^2 = 0.5197, Validation R^2 = 0.5244

Epoch 600: Train MSE = 4.8654, Validation MSE = 5.0516
Epoch 600: Train R^2 = 0.5246, Validation R^2 = 0.5301

Epoch 700: Train MSE = 4.8327, Validation MSE = 5.0112
Epoch 700: Train R^2 = 0.5278, Validation R^2 = 0.5338

Epoch 800: Train MSE = 4.8109, Validation MSE = 4.9836
Epoch 800: Train R^2 = 0.5300, Validation R^2 = 0.5364

Epoch 900: Train MSE = 4.7958, Validation MSE = 4.9641
Epoch 900: Train R^2 = 0.5314, Validation R^2 = 0.5382

Epoch 999: Train MSE = 4.7852, Validation MSE = 4.9500
Epoch 999: Train R^2 = 0.5325, Validation R^2 = 0.5395
```

Figura 12. Descenso del MSE y aumento del R^2 a lo largo de las épocas

Así que, en general, lo que se está haciendo en este ciclo es comenzar con pesos y bias inicializados en 0, realizar una predicción sobre el conjunto de entrenamiento y calcular su MSE, el cual representa el promedio del error cuadrático de las predicciones. Después, se calcula el gradiente del MSE (∇MSE), que indica en qué dirección deben modificarse los parámetros para reducir el error. Posteriormente, se realiza una predicción utilizando el conjunto de validación para comprobar qué tan bien se comporta el modelo sobre datos que no utilizó para actualizar sus parámetros. Validation no enseña nada al modelo ni modifica sus pesos; únicamente permite evaluar su desempeño durante el entrenamiento. Finalmente, se actualizan los pesos y el bias utilizando únicamente el conjunto de entrenamiento, y el ciclo se repite hasta llegar a la condición de hacer 1000 iteraciones.

Ya con la regresión lineal múltiple con gradiente descendente finalizada, y el MSE y R^2 actualizados, decidí graficar los resultados para analizar la tendencia descendente del modelo.

Aclaración: No busco que necesariamente alguna de las curvas llegue a tener un MSE de 0, ya que esto significaría que las predicciones serían perfectas sobre el conjunto evaluado o podría tratarse de una señal de que mi modelo se estaría ajustando demasiado a los datos de entrenamiento. Lo que busco es que el MSE disminuya conforme se actualicen los parámetros y que, después de cierto número de iteraciones, las mejoras sean cada vez más pequeñas. Además, es importante revisar que el comportamiento del conjunto de validación sea parecido al del conjunto de entrenamiento, ya que esto comprobaría si la mejora del modelo también funciona con los datos que no usó para actualizar sus parámetros.

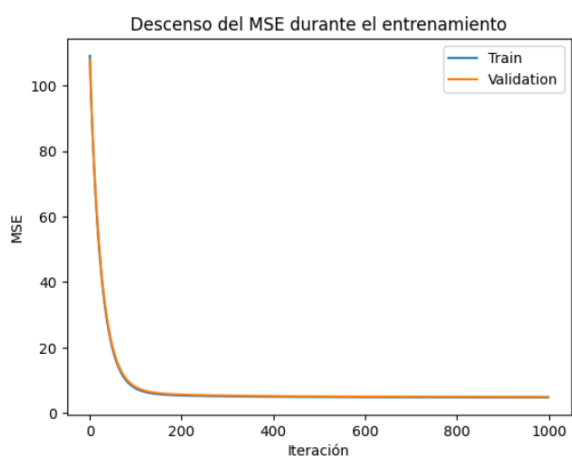


Figura 13. Gráfica del descenso del MSE a lo largo de las épocas

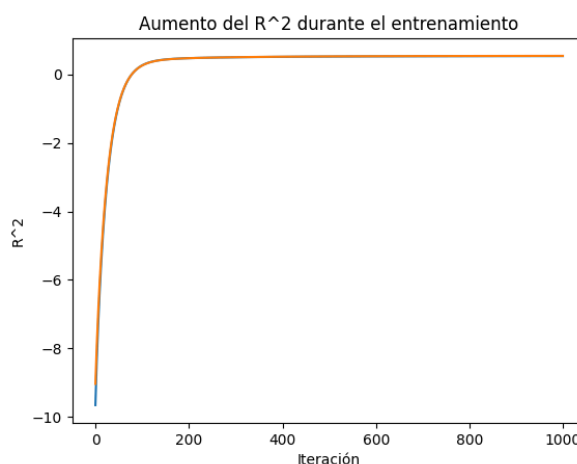


Figura 14. Gráfica del aumento del valor de R^2 a lo largo de las épocas

Efectivamente, mi modelo de regresión lineal con gradiente descendente si disminuyó el MSE tanto en el conjunto de entrenamiento como en el de validación. En el caso de R^2 , esta aumentó igualmente en los conjuntos de entrenamiento y de validación. Esto permitió que pudiera observar que el modelo estaba aprendiendo a reducir progresivamente el MSE, lo cual directamente provocaba que aumentara el R^2 , lo que indicaba que el modelo ya podía explicar una mayor proporción de la variación de Rings. Lo que más se puede destacar de estos resultados es el increíble decremento del error en las primeras 100 iteraciones, pasando de un error de 109.0606 en train a uno de 7.5505. De ahí en adelante, el error siguió disminuyendo pero a un nivel mucho menor, de modo que el MSE final que conseguí para mi conjunto de entrenamiento fue de aproximadamente 4.7852. Eso significaba que mi modelo se acercó a una zona de estabilidad. En el caso de R^2 , este también tuvo mejoras cada vez más pequeñas con cada nueva iteración, terminando con un valor de 0.5325.

Otro punto importante es que la curva de Validation tuvo una tendencia parecida a la de train tanto en la disminución de su MSE como en el aumento de la R^2 en cada iteración a un ritmo parecido. Por ejemplo, mientras que al inicio train tenía un MSE de 109.0606 y un R^2 de -9.6559 y en la iteración 100 cambió a 7.5505 y 0.2623 respectivamente, validation empezó con un MSE de 107.8676 y un R^2 de -9.0349 y en la iteración 100 pasó a tener un valor de 7.9745 y 0.2581 respectivamente. Esto significa que la mejora que se consiguió al actualizar los parámetros en train también funcionó con datos que el modelo no usó para realizar dichas actualizaciones.

En conclusión, ya pude comprobar que mi modelo aprendió. Ahora venía la parte más importante: ver si mi modelo funcionaba para predecir el resto de los registros (el 15% de los datos del conjunto de prueba) que no vió durante su etapa de entrenamiento.

5.7. Generación de predicciones al conjunto de pruebas

Como ya mencioné, ahora tocaba revisar qué tan bien funcionaba el modelo con registros que nunca había visto y que no utilizó durante su entrenamiento.

Como ya anteriormente definí la función de predicción (la función predict), pues la utilicé pero para el conjunto de pruebas.

Aclaración: Utilicé el conjunto de pruebas escalado y no el conjunto de prueba original para mantener la versión del dataset con los datos estandarizados, de modo que se mantuviera la misma escala que usó el modelo para su entrenamiento.

El resultado de mi regresión al final fue el siguiente:

```
MSE de entrenamiento: 4.7851
MSE de validación: 4.9499
MSE de prueba: 5.5383

RMSE de entrenamiento: 2.1875
RMSE de validación: 2.2248
RMSE de prueba: 2.3534

R² de entrenamiento: 0.5325
R² de validación: 0.5395
R² de prueba: 0.4861
```

Figura 15. Resultados finales de la regresión para cada conjunto de datos

Ya con la regresión aplicada a los datos de prueba, decidí generar una tabla que comparara las primeras 10 predicciones del modelo con los valores reales de Rings de dichos registros, a lo que obtuve la siguiente tabla:

	Valores Reales	Predicciones
0	12	12.61
1	7	7.44
2	9	6.40
3	7	7.19
4	8	9.62
5	10	11.18
6	9	10.20
7	18	13.65
8	6	9.51
9	6	7.05

Figura 16. Comparación de valores reales con predicciones

VI. Resultados

6.1. Predicciones contra valores reales

Una vez comparados los resultados de la predicción de mi modelo con los reales, se puede notar que algunas predicciones fueron mejores que otras. Para saber exactamente cuál fue el MSE, utilicé la función del mismo nombre que definí anteriormente (función mse()) para calcular el error cuadrático medio de mi predicción y utilicé los resultados para generar una gráfica para comparar todos los registros reales con los predichos, de modo que conseguí el siguiente resultado:

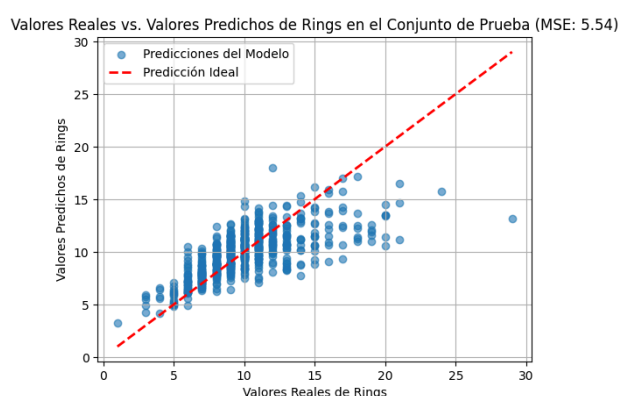


Figura 17. Comparación entre valores reales y predichos de Rings

Al final, el MSE de las predicciones de mi conjunto de pruebas fue de 5.5383, lo que significa que, en promedio, el cuadrado de la diferencia entre mis valores reales y los predichos por mi modelos fue de 5.5383. También hice lo mismo para el RMSE y para R^2 . El RMSE del conjunto de pruebas fue de 2.3534, lo que significa que la diferencia entre la cantidad de anillos predicha y real fue de aproximadamente 2.3534 anillos. El R^2 me dió un resultado de 0.4861, lo que significa que mi modelo explicaba el 48.61% de la variación de Rings para el conjunto de pruebas.

Si lo comparo con el MSE de mi conjunto de entrenamiento que fue aproximadamente de 4.7851, el MSE de mis conjuntos de validación (4.9499) y de prueba (5.5383) fueron cercanos al que obtuve durante el entrenamiento del modelo. También noté que el valor del MSE aumentaba ligeramente conforme pasé de train a validation a test. Pero como ya mencioné, la diferencia en si no fue muy grande. Eso significa que el modelo fue capaz de mantener un desempeño similar cuando trabajó con los datos con los que entrenó que

con aquellos que no usó para ajustar sus parámetros. Esto fue una buena señal de que el modelo tuvo una capacidad de generalización razonable.

Aclaración: Algo importante a notar es que el modelo devolvió predicciones con valores flotantes (o sea, con punto decimal) para los anillos cuando se supone que estos son enteros. Esto no es que esté mal o sea un error, simplemente se trata de una estimación que no es perfecta y que usa esos valores decimales para poder calcular el MSE.

Sobre la gráfica de "Valores Reales vs. Valores Predichos de Rings en el Conjunto de Prueba", tengo que la línea punteada roja representa el escenario ideal contra los puntos azules que son los valores predichos por el modelo. Algo que logré notar es que para valores muy bajos el modelo sobreestimó el valor, de modo que les predijo un valor de Rings mucho mayor al real; y para valores altos el modelo los subestimó, de modo que el modelo les predijo un valor de Rings mucho menor al real. Pero el modelo logró predecir mejor aquellos datos que estuvieron por el punto central. Así que, para las observaciones cercanas al centro de los datos el modelo predijo bien, pero se le dificultó hacer buenas estimaciones con valores extremos.

VII. Análisis y Diagnóstico

7.1. Análisis del sesgo (bias) y de la varianza

Previamente me refería al intercepto de mi modelo de regresión como bias. Para este caso, cuando hable de bias me estaré refiriendo al sesgo, que es algo diferente. Lo remarco porque no quiero que se confunda el bias que usé previamente para mi modelo de regresión lineal (intercepto) con el bias al que me voy a estar refiriendo en este punto (sesgo).

En este contexto, bias es el error sistemático que se produce cuando el modelo es demasiado simple como para poder capturar correctamente la relación que existe entre los datos.

La varianza es qué tanto cambia el modelo según los datos con los que es entrenado.

Ahora, como tal no podía calcular directamente el sesgo o la varianza. El MSE tampoco me daba directamente su valor. Lo que sí tenía eran las diferencias en los desempeños de Train, Validation y Test; las cuales podía utilizar como indicadores de cómo era el comportamiento del modelo. Estos datos los pude usar para revisar el sesgo y la varianza con estas reglas:

- Si tenía un MSE alto tanto para train como para validation, entonces eso significaría que mi modelo no estaba aprendiendo lo suficientemente bien, ni siquiera con los datos de entrenamiento. Eso podía indicar un alto sesgo o, en otras palabras, underfitting.
- Si el MSE de mi train era muy bajo pero el de mi validation era muy alto, entonces eso significaría que mi modelo funcionaba bien con los datos que usó para aprender, pero que era bastante malo cuando se trataba de recibir datos nuevos. Eso podía indicar una alta varianza o, en otras palabras, overfitting.
- En cambio, si tenía un MSE bajo tanto para train, validation y test, de modo que sus MSEs fueran parecidos, entonces esto indicaría que el modelo aprendió adecuadamente y que generalizaba correctamente. Es decir, sería consistente con un modelo que tiene un sesgo y varianza bajos. Este era el resultado ideal y el que trataba de obtener.

Lo que me decían los resultados del MSE de train (4.7851), validation (4.9499) y test (5.5383) que obtuve anteriormente es que tenía una diferencia muy pequeña entre mis MSE. Haciendo las comparaciones tuve que:

- La diferencia entre el MSE de train y validation fue de $4.9499 - 4.7851 = 0.1648$. Esta diferencia fue muy pequeña, lo que indicaba que cuando pasé de los datos utilizados para aprender a los datos que no se habían usado para actualizar los parámetros, el cambio en el rendimiento fue muy bajo.
- La diferencia entre el MSE de train y test fue de $5.5383 - 4.7851 = 0.7532$. Esta diferencia fue mayor que la de train y validation, pero siguió siendo relativamente pequeña. Eso sin contar que train, validation y test siguieron en la misma escala.

Por ende, con estas comparaciones pude concluir que no habían evidencias de un patrón donde el modelo tuviera una varianza extremadamente alta, donde el MSE de Train fuera mucho mejor que el de Validation y Test. Así que terminé con la conclusión de que no había varianza extrema en mi modelo.

Otro elemento que me pudo ayudar a respaldar esta última afirmación fue mi gráfica de "Descenso del MSE durante el entrenamiento", donde las dos curvas tuvieron un fuerte decremento en las primeras 100 iteraciones, luego sus mejoras se volvieron menores y al final terminaron muy cerca la una de la otra,

mostrando que ambas tenían comportamientos similares durante su entrenamiento.

Sobre el bias, aunque sí hubieron indicios de que podría tener cierto sesgo, no pude usar solamente mi MSE para responder si es que realmente tenía un sesgo alto. Otro dato útil fue el RMSE, ya que me permitió interpretar el error pero en la escala original de Rings, donde en mi caso obtuve un RMSE de 2.35 anillos en el conjunto de prueba. Aún así, este valor no podía decir por sí mismo si es que mi modelo presentaba un alto sesgo. Eso sin contar que la gráfica ya mostraba que mi modelo tenía problemas con ciertos valores extremos. Eso, hasta cierto punto, me hizo sentido, ya que realmente estaba usando una regresión lineal donde la relación entre las características de un abulón y su número de anillos probablemente no era del todo lineal.

Así que no podía decir que gracias a estos problemas tenía un sesgo alto. Así que, para comprobarlo, decidí que haría lo siguiente: compararía el MSE de mi modelo con un baseline, de modo que si el MSE del modelo era menor al baseline, entonces podría concluir que la regresión sí estaba aprendiendo información útil de las variables. En cambio, si el modelo apenas era un poco mejor con respecto al baseline, entonces sí podía terminar de concluir que tenía un alto sesgo.

Para hacer esta comprobación entre el MSE del modelo y el baseline, me decidí por predecir el promedio de Rings obtenido a partir de `y_train` para todos los registros.

Aclaración: La razón por la que sólo lo podía hacer con `y_train` fue porque, al tratarse de una predicción, si usaba los conjuntos `y_validation` o `y_test` estaría permitiendo que baseline conociera su información, afectando así sus predicciones.

Al final, el resultado que obtuve de esta comparación fue el siguiente:

Predicción del baseline (promedio de Rings): 9.9411			
	Conjunto	MSE del Modelo	MSE del Baseline
0	Entrenamiento	4.7851	10.2348
1	Validación	4.9499	10.7567
2	Prueba	5.5383	10.7787

Figura 18. Comparación entre el modelo y el baseline

Como el MSE del baseline fue aproximadamente el doble que el del modelo, pude concluir que tenía evidencias claras de que el modelo, al utilizar las características de abulón, generaba predicciones considerablemente mejores con respecto a simplemente predecir el promedio.

Específicamente, si tomo el caso del MSE para el conjunto de test, tengo que la reducción del MSE del modelo con respecto al baseline fue de:

$$(10.7787 - 5.5383) / 10.7787 * 100 \approx 48.618\%$$

Eso significa que mi modelo redujo el MSE del baseline un aproximado del 48.618% en el conjunto de test.

En conclusión, el modelo superó al baseline al tener un MSE menor tanto en train como en validation y test. Esto me proporcionó la evidencia necesaria como para afirmar que el modelo aprovechaba efectivamente la información de las características de los abulones para generar predicciones de Rings considerablemente mejores a tan solo usar el promedio de rings del conjunto de `y_train`. Pero me temo que, aún con esta comparación, no pude terminar de afirmar si mi modelo tenía un sesgo alto o bajo, pues el MSE del baseline como tal no representa el sesgo del modelo. Lo que sí pude terminar de concluir es que mi modelo actual realizaba mejores predicciones que otros modelos extremadamente simples que no consideren las características del abulón.

7.2. ¿Underfitting, overfitting o ajuste razonable?

Recordando lo que significa cada una de estas tres opciones:

- **Underfitting** significa que el modelo es demasiado simple como para que pueda aprender correctamente la relación entre las variables. Para afirmar que lo tenemos tendríamos que esperar un MSE alto para los MSE de train, validation y test.
- **Overfitting** significa que el modelo se adapta demasiado al conjunto de train, de modo que se esperaría un MSE muy bajo para el conjunto de train, pero muy alto para los conjuntos de validation y test.
- **Ajuste razonable** es que el modelo aprende bien las relaciones entre las variables pero no las memoriza, sino que aprende a generalizar para poder realizar correctamente predicciones con datos que no ha visto antes. Lo que se espera para concluir este resultado es un MSE bajo para train, validation y test.

De los resultados que obtuve previamente, ya sabía que el MSE de mi modelo era mejor que el de un modelo extremadamente simple que no consideraba las características de un abulón, aparte de que ya tenía

evidencia de que mi modelo sí aprendió de las relaciones entre las variables (justo por eso su MSE fue menor al del baseline). Debido a mis resultados de:

- Train: 4.7851,
- Validation: 4.9499 y
- Test: 5.5383

; no pareció haber un patrón que demostrara la presencia de overfitting, sino el MSE de mis conjuntos de validation y test tendrían que haber sido mucho mayores al de train, pero no fue el caso.

El caso del underfitting fue algo más complejo, pues que mi modelo fuera mejor que el baseline no garantizaba que estuviera aprovechando todo lo que podría aprender. Aparte, es importante recordar lo que vimos en las gráfica de que los valores extremadamente bajos eran sobreestimados por el modelo y los extremadamente altos eran subestimados; de modo que se podía apreciar cierta limitación por parte de mi modelo lineal y una tendencia de su parte a acercar sus predicciones hacia el valor medio.

Para terminar de comprobar si tenía underfitting, hice una prueba donde, como estaba trabajando con gradientes, iba a manipular la cantidad de épocas (epochs) para revisar si con más epochs el modelo seguía mejorando o si simplemente se quedaba estancado.

En esta prueba, lo que hice fue aumentar el número de épocas a 2,000. Igualmente, guardé los resultados del MSE cada 100 épocas para ver la evolución del descenso del MSE, a lo que al final del modelo obtuve los siguientes resultados:

```
Epoch 0: Train MSE = 109.0606, Validation MSE = 107.8676
Epoch 100: Train MSE = 7.5505, Validation MSE = 7.9745
Epoch 200: Train MSE = 5.3982, Validation MSE = 5.6756
Epoch 300: Train MSE = 5.1318, Validation MSE = 5.3672
Epoch 400: Train MSE = 4.9966, Validation MSE = 5.2089
Epoch 500: Train MSE = 4.9158, Validation MSE = 5.1128
Epoch 600: Train MSE = 4.8654, Validation MSE = 5.0516
Epoch 700: Train MSE = 4.8327, Validation MSE = 5.0112
Epoch 800: Train MSE = 4.8109, Validation MSE = 4.9836
Epoch 900: Train MSE = 4.7958, Validation MSE = 4.9641
Epoch 1000: Train MSE = 4.7851, Validation MSE = 4.9499
Epoch 1100: Train MSE = 4.7771, Validation MSE = 4.9391
Epoch 1200: Train MSE = 4.7711, Validation MSE = 4.9306
Epoch 1300: Train MSE = 4.7663, Validation MSE = 4.9238
Epoch 1400: Train MSE = 4.7624, Validation MSE = 4.9180
Epoch 1500: Train MSE = 4.7590, Validation MSE = 4.9130
Epoch 1600: Train MSE = 4.7561, Validation MSE = 4.9086
Epoch 1700: Train MSE = 4.7534, Validation MSE = 4.9047
Epoch 1800: Train MSE = 4.7510, Validation MSE = 4.9010
Epoch 1900: Train MSE = 4.7487, Validation MSE = 4.8976
Epoch 1999: Train MSE = 4.7466, Validation MSE = 4.8944
```

Figura 19. Valor del MSE cada 100 épocas con un total de 2,000 épocas

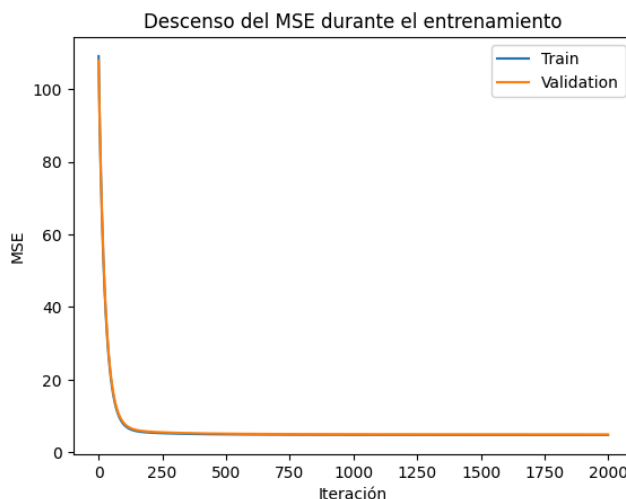


Figura 20. Descenso del MSE en el entrenamiento a lo largo de 2,000 épocas

Con este aumento de épocas, pude comprobar que técnicamente el modelo seguía aprendiendo, de modo que tanto el MSE de train como de validation seguían disminuyendo. Pero realmente estas mejoras simplemente se fueron volviendo más y más pequeñas al punto de que se volvieron casi imperceptibles, sobre todo si lo observamos visualmente en una gráfica. Aparte, no observé que hubiera un patrón donde hubiera un aumento del MSE de validation mientras que el MSE de train disminuía. Por ende, aunque con esta información no se presentó un patrón de underfitting, que no lo pudiera observar no significaba que no pueda llegar a existir. Sobre todo si contamos que el comportamiento de las predicciones extremas y de las limitaciones del modelo mostraban que podrían existir áreas de mejora.

Con esto dicho, concluyo que mis resultados no presentaron un patrón claro de underfitting.

7.3. Tuning de hiperparámetros

Previamente ya había definido los hiperparámetros manualmente, escogiendo los siguientes:

- Learning rate: 0.01
- Epochs: 1,000

Posteriormente, para probar si mi modelo tenía overfitting, modifiqué el número de épocas a 2,000.

Pero ahora venía el momento de modificar los hiperparámetros y hacerles tuning. Es decir, que el siguiente proceso iba a consistir en explorar este conjunto de opciones para ver con cuál se obtenía el mejor desempeño en validation.

De mis hiperparámetros, empecé primero a experimentar con el learning rate y mantuve mi número

de epochs de 2,000 fijo. Esto por que en el paso anterior ya había comprobado que aumentar el número de épocas, aunque permitía que el modelo siguiera mejorando, no iba a generar una mejora realmente significativa en el modelo. Así que me enfoqué en el learning rate, el cual no había modificado aún, para ver si al cambiarlo lograba que el descenso del MSE fuera más eficiente.

Me gustaría explicar una vez más lo siguiente: Al manipular el learning rate, estoy modificando qué tan grandes son los pasos que va a dar el modelo al actualizar los pesos. Tener un learning rate muy pequeño haría que obtener la solución sea muy tardado, y si fuera demasiado grande sería más difícil llegar a una convergencia debido a los grandes saltos que haría el modelo.

Entonces, probé cómo cambia el modelo con los siguientes learning rates de exploración:

- 0.001
- 0.005
- 0.01 (actual)
- 0.05
- 0.1

Con esos valores no tenía learning rates ni relativamente muy pequeños ni relativamente muy grandes, pero sí variados. Como mi learning rate actual era de 0.01, entonces lo usé como baseline para compararlo con respecto al resto de learning rates.

Lo más importante fue que, para ver efectivamente el cambio que generaba cada valor distinto del learning rate, mantuve el resto de mis parámetros igual, lo cual incluyó :

- Tener los mismos datos
- Mismos resultados en Train, Test y Validation
- Misma estandarización
- Mismos pesos iniciales
- Mismas 2,000 épocas

De modo que cada diferencia se le pudiera atribuir por completo al learning rate.

Aclaración: No utilicé el test para decidir. Para eso ya agregué Validation, de modo que para cada learning rate se obtuvo primero los pesos en train, decidí que learning rate era mejor en validation a través de comparar sus MSEs obtenidos y test se guardaría para la evaluación final. Y al final, elegí al que tenía un menor MSE en validation y no en test, pues sería aquel que le permitiría al modelo generar la mejor predicción para valores que nunca antes había visto ni usado para entrenar.

También quiero aclarar que para cada vez que hice el experimento voy tuve que reiniciar todo el estado del entrenamiento, de modo que reinicié los pesos y los bias para que empezaran con un valor de 0, y los

historiales de MSE de train y validation para que empezaran estando vacíos.

Luego de hacer las pruebas, obtuve los siguientes resultados:

	Learning Rate	Train MSE	Validation MSE	Gap
0	0.001	5.4019	5.6799	0.2780
1	0.005	4.7851	4.9500	0.1649
2	0.010	4.7466	4.8944	0.1478
3	0.050	4.6797	4.7945	0.1148
4	0.100	4.6705	4.7768	0.1063

Figura 21. Resultado del modelo con distintos learning rates

Al final, obtuve que el learning rate con los mejores resultados fue el de 0.1. Igualmente, grafiqué los resultados de modo que se pudiera contemplar visualmente la diferencia entre los resultados de cada learning rate:

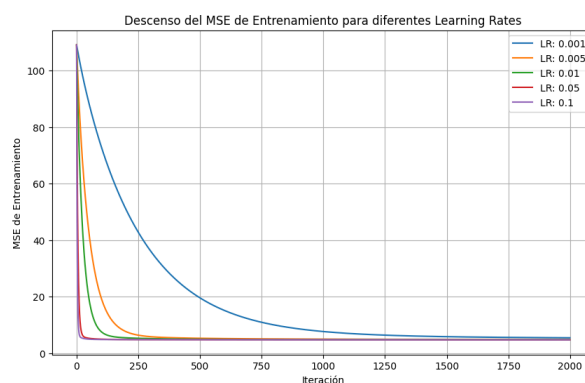


Figura 22. Descenso del MSE de entrenamiento con distintos learning rates

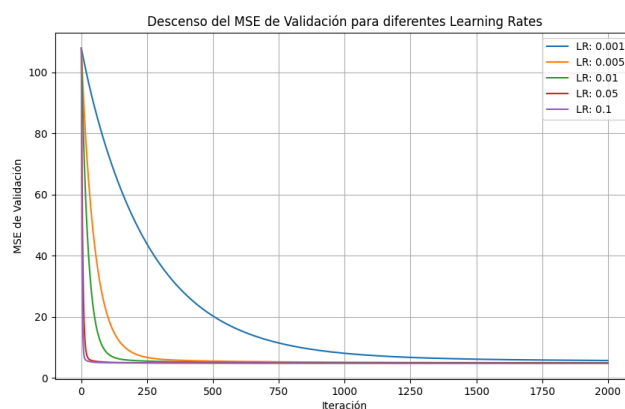


Figura 23. Descenso del MSE de validación con distintos learning rates

Sobre el tuning de las épocas, anteriormente ya probé subir el número de épocas de 1,000 a 2,000 para revisar si mi modelo tenía overfitting, y mencioné que aumentar el número de épocas sí le permitía al modelo seguir aprendiendo, pero realmente estas mejoras eran cada vez menores. Así que realmente no me pareció necesario tener que aumentar considerablemente el número de épocas si las mejoras iban a ser pequeñas.

Sobre hacer tuning del learning rate, pude conseguir varios resultados muy interesantes. Primero, se puede notar que aquel que logró un descenso más rápido del MSE fue el learning rate de 0.1 durante las primeras épocas, mientras que el último en lograrla es 0.001. Esto fue entendible debido al tamaño de los pasos en la actualización de los pesos y el bias que se daba en cada época según el learning rate.

Igualmente, el learning rate con valor de 0.1 fue el que tuvo, por una pequeña diferencia, el MSE más bajo tanto para train como para validation, seguido por el learning rate con 0.05, 0.01 (que es con el que originalmente estaba trabajando), el 0.005 y 0.001 respectivamente. También noté que, para los valores que seleccioné, el desempeño de train y de validation fue relativamente similar. Me gustaría aclarar que eso no significa que el learning rate con valor de 0.1 era la mejor opción, pero sí era la mejor para el conjunto de valores que seleccioné cuando el número de épocas era de 2,000 y mientras que el resto de componentes se mantenían constantes.

En conclusión, tener 2,000 épocas fue suficiente para entrenar el modelo. Incrementarlas sólo generaría mejores cada vez más pequeñas y costaría más recursos de lo que realmente valían la pena. Para el tuning de los learning rates, con los resultados que obtuve, al final decidí que el mejor learning rate, según el conjunto de opciones que elegí, fue el de 0.1, pues logró disminuir el MSE más rápidamente que el resto debido al tamaño de sus saltos al actualizar los pesos y el bias. Igualmente, es aquel que poseyó el menor MSE y menor diferencia entre el MSE de train y validation de todo el conjunto de opciones que seleccioné. Pero repito, aún podrían existir mejores opciones que no hubiera considerado, pero de las opciones que seleccioné esta fue la mejor.

VIII. Comparación entre el modelo original y el modelo ajustado

8.1. Modelo Original vs. Modelo Ajustado

Ahora que ya sabía que había una mejor combinación de hiperparámetros para mi modelo, decidí comparar los resultados del modelo con learning rate de 0.1 (que fue el que dió los mejores resultados del conjunto de opciones que evalué) con mi modelo original, el cual tenía un learning rate de 0.01. El objetivo fue ver cuál de los dos modelos podía predecir con menor margen de error el valor de los anillos de un abulón, y también ver cuál poseía una mayor proporción de la variación.

Al hacer dicha comparación, obtuve los siguientes resultados (Me gustaría aclarar que la tabla de evidencia está transpuesta, ya que la original fue demasiado larga como para caber, pero la del código original no está transpuesta):

Modelo	Original	Ajustado
Learning Rate	0.01	0.1
Train MSE	4.746553	4.670542
Validation MSE	4.89439	4.776771
Test MSE	5.453579	5.182247
Train RMSE	2.178658	2.161144
Validation RMSE	2.212327	2.185582
Test RMSE	2.33529	2.276455
Train R ²	0.536234	0.543661
Validation R ²	0.544677	0.555619
Test R ²	0.493927	0.519105

Figura 24. Modelo original vs. Modelo ajustado

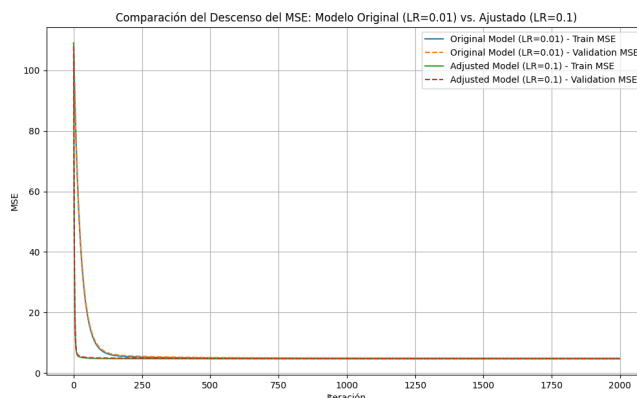


Figura 25. Descenso del MSE del modelo original y del modelo ajustado

Luego de comparar ambos modelos, observé que los valores de MSE en Train, Validation y Test fueron

menores en el modelo ajustado que tuvo el learning rate de 0.1 que en el modelo original con el learning rate de 0.01. Lo mismo pasó con el RMSE, que fue ligeramente menor en el modelo ajustado.

Al hacer la comparación directa, basándome en el MSE del conjunto de pruebas, obtuve que el modelo ajustado disminuyó el MSE en un 4.98% con respecto al modelo original, de modo que el ajuste del learning rate sí generó una mejora al modelo, aún si dicha mejora fue pequeña.

En conclusión, bajo las condiciones en las que los evalué, el modelo ajustado logró obtener mejores resultados que el modelo original, aunque las mejoras fueron relativamente pequeñas, de modo que el modelo ajustado será la versión final de mi modelo.

Aclaración: Que el modelo ajustado ($\alpha = 0.1$) haya obtenido mejores resultados que mi modelo original ($\alpha = 0.01$) no implica que sea el mejor modelo para resolver este problema. Simplemente fue la mejor opción entre los valores de learning rate que seleccioné y probé.

Más adelante, realicé otras dos gráficas más para comparar los valores del RMSE y del R^2 para cada uno de los conjuntos de datos en cada uno de los modelos, a lo que obtuve los siguiente resultados:

Igualmente hice la comparación directa, también basándome en el conjunto de pruebas, del RMSE y de R^2 entre los modelos para ver cuál fue el porcentaje de mejora.

Para el RMSE del conjunto de pruebas, obtuve un valor de 2.3353 para el modelo original, mientras que para el modelo ajustado obtuve un valor de 2.2765. Eso significa, para el modelo ajustado, que la diferencia entre la cantidad de anillos predicha y real fue de aproximadamente 2.2765 anillos mientras que en la original fue una diferencia de 2.3353 anillos. Al hacer los cálculos en python, obtuve que el porcentaje de mejora del modelo ajustado con respecto al modelo original fue de 2.52%. Una mejora ligera pero que aún así hace que el resultado del modelo ajustado sea más preciso.

Para el R^2 del conjunto de prueba, en el modelo original obtuve un valor de 0.4939 mientras que el modelo ajustado obtuve un valor de 0.5191. Eso significa que el modelo ajustado explica aproximadamente el 51.91% de la varianza de Rings, mientras que el modelo original sólo explica el 49.39%. Al hacer los cálculos en python, obtuve que el porcentaje de mejora del modelo ajustado con respecto al modelo original fue de -5.10%. La razón por la que el valor salió negativo fue porque en este

caso el valor de R^2 subió, no bajó. Pero eso es exactamente lo que esperaba, ya que eso significa que el modelo ajustado puede explicar de forma más precisa la proporción de la varianza de Rings en un aproximado de 5.10%.

En conclusión, al modificar el hiperparámetro del learning rate y encontrar un valor más eficiente, logré hacer que el modelo tuviera una mejora significativa aunque pequeña a la hora de realizar predicciones acerca del número de anillos de un abulón. Al final, este cambio permitió reducir el MSE y, por ende, aumentar la proporción de la varianza de Rings que es explicada por el modelo. Así que, para los modelos que trabajé, el que consiguió los mejores resultados fue el modelo ajustado, de modo que ese terminó siendo mi modelo final. Repito, eso no significa que sea la mejor opción, pero sí es la mejor opción para los parámetros con los que trabajé.

IX. Regresión Lineal con Mínimos Cuadrados Ordinarios (OLS)

Previamente ya había logrado generar un modelo de regresión lineal con gradiente descendente optimizado para predecir el número de anillos de un abulón. Pero todo esto lo hice sin frameworks o bibliotecas especializadas en Machine Learning, por lo que tuve que implementar manualmente varias funcionalidades y hacer pruebas manuales para revisar que funcionaran. Igualmente, tuve que hacer pruebas de tuning de hiperparámetros, comparar modelos y escoger la mejor combinación para tener la versión más optimizada de mi modelo. Así que ahora venía la pregunta: ¿Cómo hubiera sido todo este proceso de crear y entrenar un modelo de regresión usando frameworks? Es aquí donde entra lo que será esta sección.

Para la versión del modelo con framework, seguí utilizando el modelo de regresión lineal múltiple para predecir el valor de Rings, solo que esta vez usaría la biblioteca de scikit-learn en vez de implementar todo el algoritmo manualmente. Procesos como el de ETL, la limpieza de datos y el análisis de correlación fueron exactamente iguales a los que hice para la versión sin framework, pues ambas versiones trabajaban con el mismo dataset que previamente ya había procesado.

9.1. ¿Qué es OLS?

A diferencia de la versión del modelo con framework donde usé Gradiente Descendente para actualizar iterativamente los valores de los pesos y del bias, el LinearRegression de la biblioteca scikit-learn básicamente me ayudó a resolver la regresión a través de usar Mínimos Cuadrados Ordinarios (OLS).

Lo que hace OLS es que busca los pesos óptimos en un solo paso matemático, en lugar de irlos aproximando iterativamente. La fórmula clásica para obtener para obtener estos pesos es la de la Ecuación Normal:

$$w = (X^T X)^{-1} X^T y$$

Donde:

- X = matriz de las características del abulón.
- X^T = matriz transpuesta del abulón.
- y = vector de los valores reales de Rings.
- w = vector de pesos que minimiza la MSE.

Técnicamente scikit-learn no hace este cálculo directamente. En lugar de eso, el LinearRegression calcula estos pesos a través de la descomposición de valores singulares (SVD). Lo que hace SVD es que descompone cualquier matriz (en este caso del dataset) en matrices más simples para que cada parte muestre una parte distinta de los datos originales, como las correlaciones o los pesos.

De momento, el cambio más grande que habría con respecto a mi versión del modelo sin frameworks es que aquí no van a haber ni learning rates ni épocas. Esto no es necesario ya que, como mencioné previamente, OLS no aprende iterativamente para acercarse al valor mínimo del MSE, hace un solo cálculo que consigue directamente los valores de los pesos que minimizan el error del conjunto de train.

X. Comparación entre el modelo sin framework y el modelo con framework

8.1. Modelo Original vs. Modelo Ajustado

Ahora q

XI. Conclusiones

El modelo de regresión lineal fue una herramienta muy útil para predecir el número de anillos de un conjunto de abulones, donde al final logré hacer que mi modelo pudiera explicar aproximadamente el 51.91% de la varianza de los anillos, consiguiendo resultados similares a los reales para los datos más posicionados en el centro. Admito que el modelo aún tiene áreas de mejora, sobre todo cuando se trata de predicciones con valores más extremos. Pero gracias al ajuste de los hiperparámetros y a la realización de pruebas y comparaciones de los distintos modelos puede conseguir una versión más optimizada y precisa a la que tenía originalmente, el cual terminó con un buen rendimiento para trabajar no sólo con conjuntos de entrenamiento y de validación, sino también para conjuntos de pruebas sin sufrir undefitting y sin presentar patrones de overfitting.

Referencias bibliográficas

- [1] Ashok Kumar Palivela (2021). “Predicción de la edad del abulón”. Recuperado de: <http://kaggle.com/code/ashokkumarpalivela/abalone-age-prediction>
- [2] Harshitha Indumathi (2021). “Predicción de la edad del abulón”. Recuperado de: <https://harshithaindumathi.medium.com/abalone-age-prediction-607ecda3fa3b>
- [3] Brayan Buitrago (2020). “Aprendizaje automático — Modelos de Regresión I”. Recuperado de: <https://medium.com/iwannabedatadriven/machine-learning-modelos-de-regresi%C3%B3n-i-d293ae235e9a>
- [4] Khan Academy (S.F.). “Descenso de gradiente”. Recuperado de: <https://es.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-derivatives/optimizing-multivariable-functions/a/what-is-gradient-descent>
- [5] AWS (2022). “¿Qué es extracción, transformación y carga (ETL)?”. Recuperado de: <https://aws.amazon.com/es/what-is/etl/>
- [6] Informatica (S.F.). “¿Qué es ETL (extracción, transformación y carga)?”. Recuperado de: <https://www.informatica.com/resources/articles/what-is-etl.html>
- [7] GeeksForGeeks (2026). “Codificación One-Hot en Aprendizaje Automático”. Recuperado de: <https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/>

- [8] Vinod Chugani (2025). “Codificación One-Hot: Entendiendo lo “caliente” en los datos”. Recuperado de:
<https://machinelearningmastery.com/one-hot-encoding-understanding-the-hot-in-data/>
- [9] Michael DelSole (2018). “¿Qué es la codificación One Hot y cómo se realiza?”. Recuperado de:
<https://medium.com/@michaeldelsole/what-is-one-hot-encoding-and-how-to-do-it-f0ae272f1179>
- [10] jmp Statistical Discovery (S.F.). “Correlación”. Recuperado de:
<https://medium.com/@michaeldelsole/what-is-one-hot-encoding-and-how-to-do-it-f0ae272f1179>
- [11] Máxima formación (2020). “¿Qué es la correlación estadística y cómo interpretarla?”. Recuperado de:
<https://www.maximaformacion.es/blog/que-es-la-correlacion-estadistica-y-como-interpretarla>
- [12] Mirko Stojiljković (2019). “NumPy, SciPy y pandas: correlación con Python”. Recuperado de:
<https://realpython.com/numpy-scipy-pandas-correlation-python/>
- [13] Eda Kavlakoglu y Jacob Murel Ph.D. (S.F.). “¿Qué es la multicolinealidad?”. Recuperado de:
<https://www.ibm.com/mx-es/think/topics/multicollinearity>
- [14] Máxima Formación (2022). “¿Qué es la multicolinealidad y por qué es un problema?”. Recuperado de:
<https://www.maximaformacion.es/blog/que-es-la-multicolinealidad-y-por-que-es-un-problema>
- [15] Google for developers (S.F.). “Regresión lineal: Hiperparámetros”. Recuperado de:
<https://developers.google.com/machine-learning/crash-course/linear-regression/hyperparameters?hl=es-419>
- [16] Priyansh Soni (2024). “Hiperparámetros en el aprendizaje automático explicados”. Recuperado de:
<https://www.blog.trainindata.com/hyperparameters-in-machine-learning/>
- [17] Ivan Belcic y Cole Stryker (S.F.). “¿Qué es el ajuste de hiperparámetros?”. Recuperado de:
<https://www.ibm.com/mx-es/think/topics/hyperparameter-tuning>
- [18] AWS (S.F.). “¿En qué consiste el ajuste de hiperparámetros?”. Recuperado de:
<https://aws.amazon.com/es/what-is/hyperparameter-tuning/>
- [19] MATLAB (S.F.). “Coeficiente de determinación (R^2)”. Recuperado de:
<https://la.mathworks.com/help/stats/coefficient-of-determination-r-squared.html>
- [20] Scikit learn (S.F.). “Ordinary Least Squares and Ridge Regression”. Recuperado de:
https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols_ridge.html
- [21] Dario Radečić (2026). “Descomposición en valores irregulares (SVC): lo que debes saber”. Recuperado de:
<https://www.datacamp.com/es/tutorial/singular-value-decomposition>
- [22]